Arrow vs Regular

Not just shorter: they treat this differently

	Arrow	Regular
Own this	✗ no	✓ yes
Own arguments	✗ no	✓ yes
Use with new	✗ no	✓ yes
Short return	✓ yes	✗ no

1 Arrow keeps this

```
const o = { n: 2 };
o.f = function () {
  return () => this.n;
};
const h = o.f();
h();   // 2
```

2 Class field arrow

```
class B {
  m() {}
  handle = () => this;
}
const a = new B(), b = new B();
a.m === b.m;   // true
a.handle === b.handle; // false
```

TRAP What does this log?

```
const a = () => { n: 1 };
const b = () => ({ n: 1 });
console.log(a(), b());
```

`undefined` `{ n: 1 }`

{} after **=>** is a code block. Wrap an object in **()**.

KEY TAKEAWAY

- `arrow` this comes from outside
- `function` Own this, arguments, new
- `({ })` Wrap a returned object

? Follow-up questions to expect

- **Why can't an arrow be a constructor?** → No own this and no prototype, so new throws.
- **Where would you avoid arrows?** → Object methods and constructors.
- **Can bind or call change an arrow's `this`?** → No. They ignore it.
- **Is an arrow hoisted like a declaration?** → No. It lives in a const, so early use throws.